

Министерство науки и высшего образования Российской Федерации

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ «Национальный исследовательский университет ИТМО»
(Университет ИТМО)**

Факультет прикладной информатики

Образовательная программа Мобильные и сетевые технологии

Направление подготовки 09.03.03 Прикладная информатика

О Т Ч Е Т

по лабораторной работе №6.2

Тема задания: «Работа с БД в СУБД MongoDB»

Обучающийся: Степанова Екатерина Андреевна, К3241

Руководитель: Говорова Марина Михайловна

Белов Александр Олегович

Санкт-Петербург,
2026

Цель: овладеть практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

Оборудование: компьютерный класс.

Программное обеспечение: СУБД MongoDB 8.2.

ВЫПОЛНЕНИЕ

2 CRUD-ОПЕРАЦИИ В СУБД MONGODB. ВСТАВКА ДАННЫХ. ВЫБОРКА ДАННЫХ

2.1 ВСТАВКА ДОКУМЕНТОВ В КОЛЛЕКЦИЮ

Практическое задание 2.1.1.

- 1) Создадим базу данных learn

Сделать это можно с помощью команды use.

```
test> use learn
switched to db learn
```

Рис. 1 - Создание бд learn

- 2) заполним коллекцию единорогов unicorns предложенными данными

```
learn> db.unicorns.insert({name: 'Horny', loves: ['carrot', 'papaya'], weight
: 600, gender: 'm', vampires: 63});
| db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 45
0, gender: 'f', vampires: 43});
| db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'], weight
: 984, gender: 'm', vampires: 182});
| db.unicorns.insert({name: 'Roooooodles', loves: ['apple'], weight: 575, ge
gender: 'm', vampires: 99});
| db.unicorns.insert({name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'
], weight: 550, gender: 'f', vampires: 80});
| db.unicorns.insert({name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 7
33, gender: 'f', vampires: 40});
| db.unicorns.insert({name: 'Kenny', loves: ['grape', 'lemon'], weight: 690,
gender: 'm', vampires: 39});
| db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 42
1, gender: 'm', vampires: 2});
| db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight:
601, gender: 'f', vampires: 33});
| db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight:
650, gender: 'm', vampires: 54});
| db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540
, gender: 'f'});
|
DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insert
Many, or bulkWrite.
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a0ca415de48b0ea48abc11e') }
}
```

Рис. 2 - заполнение коллекции данными

Я хочу проверить, что коллекция действительно создалась и посмотреть ее содержимое. Сделать это можно с помощью команд

show collections и *db.unicorns.find()*, соответственно.

```
learn> show collections
unicorns
learn> db.unicorns.find()
[
  {
    _id: ObjectId('6a0ca415de48b0ea48abc114'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a0ca415de48b0ea48abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a0ca415de48b0ea48abc116'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
```

Рис. 3 - Проверка коллекции unicorns

- 3) Используя второй способ вставки, который был представлен в лабораторной работе, проведем операцию.

Второй способ добавления в бд документа включает два этапа: определение документа (`document = ( { ... } )`) и собственно его добавление:

```
> document=({"name": "Bill", "age": 32, "languages": ["english", "french"]})
```

```
> db.users.insert(document)
```

```

gender: 'm', vampires: 165}
learn> document={name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704,
gender: 'm', vampires: 165}
| )
{
  name: 'Dunx',
  loves: [ 'grape', 'watermelon' ],
  weight: 704,
  gender: 'm',
  vampires: 165
}

```

Рис. 4 - Определение документа

```

learn> db.unicorns.insert(document)
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a0ca612de48b0ea48abc11f') }
}

```

Рис. 5 - Вставка документа

Проверим, что вставка выполнена, выведем содержимое коллекции unicorns. Найдем точно по имени.

```

learn> db.unicorns.find({name : "Dunx"})
[
  {
    _id: ObjectId('6a0ca612de48b0ea48abc11f'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a0ca68dde48b0ea48abc120'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  }
]

```

Рис. 6 - Просмотр содержимого коллекции unicorns

Еще я заметила, что случайно добавила этого единорога 2 раза, удалим одного из них, например с `_id = 6a0ca612de48b0ea48abc11f`.

Сделать это можно с помощью команды:

```
db.unicorns.deleteOne({_id: '6a0ca612de48b0ea48abc11f'})
```

```
learn> db.unicorns.deleteOne({_id: ObjectId('6a0ca612de48b0ea48abc11f')})  
{ acknowledged: true, deletedCount: 1 }
```

Рис. 7 - Удаление документа по oid

Теперь снова из коллекции выведем единорога с именем Dunx

```
learn> db.unicorns.find({name: 'Dunx'})  
[  
  {  
    _id: ObjectId('6a0ca68dde48b0ea48abc120'),  
    name: 'Dunx',  
    loves: [ 'grape', 'watermelon' ],  
    weight: 704,  
    gender: 'm',  
    vampires: 165  
  }  
]
```

Рис. 8 - Просмотр содержимого документа по фильтру

4) Проверим содержимое коллекции с помощью find

```
learn> db.unicorns.find()
[
  {
    _id: ObjectId('6a0ca415de48b0ea48abc114'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a0ca415de48b0ea48abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a0ca415de48b0ea48abc116'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a0ca415de48b0ea48abc117'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a0ca415de48b0ea48abc118'),
```

Рис. 9 - Просмотр содержимого коллекции unicorns

2.2 ВЫБОРКА ДАННЫХ ИЗ БД

Практическое задание 2.2.1.

- 1) Сформируйте запросы для вывода списков самцов и самок единорогов. Ограничьте список самок первыми тремя особями. Отсортируйте списки по имени.

Список самцов, отсортированный по имени.

```
learn> db.unicorns.find({gender:'m'}).sort({name : 1})
[
  {
    _id: ObjectId('6a0ca68dde48b0ea48abc120'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a0ca415de48b0ea48abc114'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a0ca415de48b0ea48abc11a'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a0ca415de48b0ea48abc11d'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],

```

Рис. 10 - Просмотр содержимого документа по фильтру

Список самок, отсортированный по имени и ограниченный первыми тремя особями.


```
learn> db.unicorns.find({gender:'f'}).sort({name : 1}).limit(3)
[
  {
    _id: ObjectId('6a0ca415de48b0ea48abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a0ca415de48b0ea48abc119'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a0ca415de48b0ea48abc11c'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  }
]
```

Рис. 11 - Просмотр содержимого документа по фильтру

- 2) Найдите всех самок, которые любят carrot. Ограничьте этот список первой особью с помощью функций findOne и limit.

```
learn> db.unicorns.findOne({gender:'f', loves : 'carrot'})
{
  _id: ObjectId('6a0ca415de48b0ea48abc115'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
```

Рис. 12 - Просмотр содержимого документа по фильтру

Однако, в данном случае это не совсем корректный запрос, потому что к методу findOne прибегают в случае, когда ожидается, что результирующая коллекция будет содержать лишь один документ. В нашем случае, если вывести все документы, удовлетворяющих условию, то получится 3 записи.

```
learn> db.unicorns.find({gender:'f', loves : 'carrot'})
[
  {
    _id: ObjectId('6a0ca415de48b0ea48abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a0ca415de48b0ea48abc118'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a0ca415de48b0ea48abc11e'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```

Рис. 13 - Просмотр содержимого документа по фильтру

Поэтому в данном случае лучше использовать limit.

```
learn> db.unicorns.find({gender:'f', loves : 'carrot'}).limit(1)
[
  {
    _id: ObjectId('6a0ca415de48b0ea48abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
```

Рис. 14 - Просмотр содержимого документа по фильтру

Если же мы хотим выводить только конкретные поля, то можно воспользоваться этим правилом.

По умолчанию выборка содержит все поля документа, однако, в том случае, если требуется выбрать лишь конкретные поля, методам find() и findOne() можно передавать второй аргумент в виде JSON-структуры, с ключами, совпадающими с названиями столбцов

и значениями 1, если поле должно попадать в выборку и 0, если его необходимо исключить из выборки.

Вывести только значения полей "age" у все документов, в которых name=Tom:

```
> db.users.find({name: "Tom"}, {age: 1})
```

Использование единицы в качестве параметра {age: 1} указывает, что запрос должен вернуть только содержание свойства age.

Обратная ситуация: нужно найти все параметры документа, кроме свойства age. В этом случае в качестве параметра указать 0:

```
> db.persons.find({name: "Tom"}, {age: 0})
```

При этом надо учитывать, что даже если мы отметим, что мы хотим получить только поле name, поле _id также будет включено в результирующую выборку. Поэтому, если не нужно видеть данное поле в выборке, то надо явным образом указать: {"_id":0}

Альтернативно вместо 1 и 0 можно использовать true и false:

```
> db.users.find({name: "Tom"}, {age: true, _id: false})
```

Давайте попробуем на примере, допустим скроем поле gender, а все остальные выведем.

```
learn> db.unicorns.find({gender: 'f', loves : 'carrot'}, {gender: 0}).limit(1)
[
  {
    _id: ObjectId('6a0ca415de48b0ea48abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    vampires: 43
  }
]
```

Рис. 15 - Просмотр содержимого документа по фильтру

Или же захотим наоборот скрыть все, кроме имени.

```
learn> db.unicorns.find({gender:'f', loves: 'carrot'}, {name: 1}).limit(1)
[ { _id: ObjectId('6a0ca415de48b0ea48abc115'), name: 'Aurora' } ]
```

Рис. 15 - Просмотр содержимого документа по фильтру

Практическое задание 2.2.2.

- 1) Модифицируйте запрос для вывода списков самцов единорогов, исключив из результата информацию о предпочтениях и поле.

```
learn> db.unicorns.find({gender:'m'}, {loves: false, gender: false})
[
  {
    _id: ObjectId('6a0ca415de48b0ea48abc114'),
    name: 'Horny',
    weight: 600,
    vampires: 63
  },
  {
    _id: ObjectId('6a0ca415de48b0ea48abc116'),
    name: 'Unicrom',
    weight: 984,
    vampires: 182
  },
  {
    _id: ObjectId('6a0ca415de48b0ea48abc117'),
    name: 'Roooooodles',
    weight: 575,
    vampires: 99
  },
  {
    _id: ObjectId('6a0ca415de48b0ea48abc11a'),
    name: 'Kenny',
    weight: 690,
    vampires: 39
  },
  {
    _id: ObjectId('6a0ca415de48b0ea48abc11b'),
```

Рис. 16 - Просмотр содержимого документа по фильтру

Практическое задание 2.2.3.

- 1) Вывести список единорогов в обратном порядке добавления. Если нужно отсортировать ограниченную коллекцию, то можно воспользоваться параметром \$natural. Этот параметр позволяет задать сортировку: документы передаются в том порядке, в каком они были добавлены в коллекцию, либо в обратном порядке.

```
learn> db.unicorns.find().sort({$natural:-1})
[
  {
    _id: ObjectId('6a0ca68dde48b0ea48abc120'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a0ca415de48b0ea48abc11e'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a0ca415de48b0ea48abc11d'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a0ca415de48b0ea48abc11c'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
  }
]
```

Рис. 17 - Просмотр содержимого документа с сортировкой

Практическое задание 2.2.4.

- 1) Вывести список единорогов с названием первого любимого предпочтения, исключив идентификатор.

```
learn> db.unicorns.find({}, {_id : 0, loves : {$slice: 1}})
[
  {
    name: 'Horny',
    loves: [ 'carrot' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    name: 'Aurora',
    loves: [ 'carrot' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    name: 'Unicrom',
    loves: [ 'energon' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    name: 'Solnara',
    loves: [ 'apple' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
]
```

Рис. 18 - Просмотр содержимого документа с названием первого любимого предпочтения, исключив идентификатор

2.3 ЛОГИЧЕСКИЕ ОПЕРАТОРЫ

Практическое задание 2.3.1.

- 1) Вывести список самок единорогов весом от полутонны до 700 кг, исключив вывод идентификатора.

```
learn> db.unicorns.find({gender: 'f', weight: {$gt: 500, $lt: 700}})
[
  {
    _id: ObjectId('6a0ca415de48b0ea48abc118'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a0ca415de48b0ea48abc11c'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId('6a0ca415de48b0ea48abc11e'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```

Рис. 19 - Результат выполнения

Практическое задание 2.3.2.

- 1) Вывести список самцов единорогов весом от полутонны и предпочитающих grape и lemon, исключив вывод идентификатора.

```
learn> db.unicorns.find({gender: 'm', loves: {$all : ["grape", "lemon"]}, weight: {$gt: 500, $lt: 700}}, {_id: 0})
[
  {
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]
```

Рис. 20 - Результат выполнения

Практическое задание 2.3.3.

- 1) Найти всех единорогов, не имеющих ключ vampires.

```
learn> db.unicorns.find({vampires: {$exists: false}})
[
  {
    _id: ObjectId('6a0ca415de48b0ea48abc11e'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```

Рис. 21 - Результат выполнения

Практическое задание 2.3.4.

- 1) Вывести список упорядоченный список имен самцов единорогов с информацией об их первом предпочтении.


```

MongoServerError: [BadValue] unknown operator: $slice
learn> db.unicorns.find({}, {loves: {$slice : 1}})
[
  {
    _id: ObjectId('6a0ca415de48b0ea48abc114'),
    name: 'Horny',
    loves: [ 'carrot' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a0ca415de48b0ea48abc115'),
    name: 'Aurora',
    loves: [ 'carrot' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a0ca415de48b0ea48abc116'),
    name: 'Unicrom',
    loves: [ 'energon' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a0ca415de48b0ea48abc117'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a0ca415de48b0ea48abc118'),

```

Рис. 22 - Результат выполнения

**3 ЗАПРОСЫ К БАЗЕ ДАННЫХ MONGODB.
ВЫБОРКА ДАННЫХ. ВЛОЖЕННЫЕ ОБЪЕКТЫ.
ИСПОЛЬЗОВАНИЕ КУРСОРОВ. АГРЕГИРОВАННЫЕ ЗАПРОСЫ.
ИЗМЕНЕНИЕ ДАННЫХ**

3.1 ЗАПРОС К ВЛОЖЕННЫМ ОБЪЕКТАМ

Практическое задание 3.1.1.

- 1) Создайте коллекцию towns, включающую следующие документы:

```
{name: "Punxsutawney ",  
  populatiuon: 6200,  
  last_sensus: ISODate("2008-01-31"),  
  famous_for: [""],  
  mayor: {  
    name: "Jim Wehrle"  
  }}  
{name: "New York",  
  populatiuon: 22200000,  
  last_sensus: ISODate("2009-07-31"),  
  famous_for: ["status of liberty", "food"],  
  mayor: {  
    name: "Michael Bloomberg",  
    party: "I"}}  
{name: "Portland",  
  populatiuon: 528000,  
  last_sensus: ISODate("2009-07-20"),  
  famous_for: ["beer", "food"],  
  mayor: {  
    name: "Sam Adams",  
    party: "D"}}
```

Создадим коллекцию towns с помощью команды

`db.createCollection("towns")`

```
learn> db.createCollection('towns')  
{ ok: 1 }  
learn> show collections  
towns  
unicorns
```

Рис. 23 - Создание коллекции towns

Вставим данные с помощью команды insert. Ниже представлен результат выполнения.

```
learn> db.towns.find()
[
  {
    _id: ObjectId('6a0cbb24de48b0ea48abc121'),
    name: 'Punxsutawney ',
    populatiuon: 6200,
    last_sensus: ISODate('2008-01-31T00:00:00.000Z'),
    famous_for: [ '' ],
    mayor: { name: 'Jim Wehrle' }
  },
  {
    _id: ObjectId('6a0cbb53de48b0ea48abc122'),
    name: 'New York',
    populatiuon: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a0cbb69de48b0ea48abc123'),
    name: 'Portland',
    populatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
```

Рис. 24 - Результат вставки в коллекцию towns

- 2) Сформировать запрос, который возвращает список городов с независимыми мэрами (party="I"). Вывести только название города и информацию о мэре.

```
3) db.towns.find({"mayor.party" : "I"},{name: 1, mayor: 1})
4) [
5)   {
6)     _id: ObjectId('6a0cbb53de48b0ea48abc122'),
7)     name: 'New York',
8)     mayor: { name: 'Michael Bloomberg', party: 'I' }
9)   }
10) ]
```

```
learn> db.towns.find({"mayor.party" : "I"},{name: 1, mayor: 1})
[
  {
    _id: ObjectId('6a0cbb53de48b0ea48abc122'),
    name: 'New York',
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  }
]
```

Рис. 25 - Вывод документа при заданном условии

Если поле `_id` тоже не нужно показывать, то

```
learn> db.towns.find({"mayor.party" : "I"},{name: 1, mayor: 1, _id : 0})
[
  { name: 'New York', mayor: { name: 'Michael Bloomberg', party: 'I' } }
]
```

Рис. 26 - Вывод документа при заданном условии

3) Сформировать запрос, который возвращает список беспартийных мэров (party отсутствует). Вывести только название города и информацию о мэре.

```
1. db.towns.find({"mayor.party" : {$exists: false}}, {name: 1, mayor:
  1, _id : 0})
```

```
learn> db.towns.find({"mayor.party" : {$exists: false}}, {name: 1, mayor: 1,
_id : 0})
[ { name: 'Punxsutawney ', mayor: { name: 'Jim Wehrle' } } ]
```

Рис. 27 - Вывод документа при заданном условии

Практическое задание 3.1.2.

1) Сформировать функцию для вывода списка самцов единорогов.

```
learn> function findMale() {return db.unicorns.find({gender: "m"})}
[Function: findMale]
```

Рис. 28 - Создание функции

```

learn> findMale()
[
  {
    _id: ObjectId('6a0ca415de48b0ea48abc114'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a0ca415de48b0ea48abc116'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a0ca415de48b0ea48abc117'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
  }
]

```

Рис. 29 - Выполнение функции

- 2) Создать курсор для этого списка из первых двух особей с сортировкой в лексикографическом порядке.

```
var cursor_1 = db.unicorns.find({gender: 'm'}); null;
```

```

learn> var cursor_1 = db.unicorns.find({gender: 'm'}); null;
null

```

Рис. 30 - Создание курсора

```
cursor_1.sort({name: 1}).limit(2)
```

```
learn> cursor_1.sort({name: 1}).limit(2)
[
  {
    _id: ObjectId('6a0ca68dde48b0ea48abc120'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a0ca415de48b0ea48abc114'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  }
]
```

Рис. 31- Результат выполнения

3) Вывести результат, используя forEach

Так как первые выводы у курсора уже состоялись, пришлось создать снова и вывести с помощью цикла.

```
cursor.forEach(function(obj) { print(obj.name); });
```

```
learn> var cursor = db.unicorns.find({gender: 'm'}).sort({name: 1}).limit(2)
; null;
null
learn> cursor.forEach(function(obj) { print(obj.name); });
Dunx
Horny
```

Рис. 32 - Результат выполнения

3.2 АГРЕГИРОВАННЫЕ ЗАПРОСЫ

Практическое задание 3.2.1.

1) Вывести количество самок единорогов весом от полутонны до 600

кг.

```
db.unicorns.find({gender: "f", weight: {$lt: 600}})
```

Выведем все документы по данному условию, а далее применим count для подсчета самок, удовлетворяющих условию.

```
learn> db.unicorns.find({gender: "f", weight: {$lt: 600}})
[
  {
    _id: ObjectId('6a0ca415de48b0ea48abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a0ca415de48b0ea48abc118'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a0ca415de48b0ea48abc11e'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn> db.unicorns.find({gender: "f", weight: {$lt: 600}}).count()
3
```

Рис. 33 - Результат выполнения

Практическое задание 3.2.2.

- 1) Вывести список предпочтений.

```
db.unicorns.distinct("loves")
```

```
learn> db.unicorns.distinct("loves")
[
  'apple',      'carrot',
  'chocolate', 'energon',
  'grape',      'lemon',
  'papaya',     'redbull',
  'strawberry', 'sugar',
  'watermelon'
]
```

Рис. 34 - Результат выполнения

Практическое задание 3.2.3.

- 1) Подсчитать количество особей единорогов обоих полов.

```
db.unicorns.aggregate({"$group":{"_id": "$gender", count:{$sum: 1}}})
```

Пояснение: Параметр `_id` указывает, что группировка будет производиться по ключу `gender`. Значение параметра `$sum` инициализирует начальное значение поля `count`. Это поле будет представлять количество элементов для группы.

```
learn> db.unicorns.aggregate({"$group":{"_id": "$gender", count:{$sum: 1}}})
[ { _id: 'm', count: 7 }, { _id: 'f', count: 5 } ]
```

Рис. 35 - Результат выполнения

3.3 РЕДАКТИРОВАНИЕ ДАННЫХ

Практическое задание 3.3.1.

- 1) Выполнить команду:

```
> db.unicorns.save({name: 'Barney', loves: ['grape'], weight: 340, gender: 'm'})
```

Метод `save` является устаревшим для новых версий MongoDB.

Так как у меня в коллекции нет документа с именем 'Barney', то я его просто добавлю.

В ином случае, если бы было необходимо обновить данные, то нужно использовать метод `updateOne()` или `updateMany()`


```
learn> db.unicorns.insert({name: 'Barney', loves: ['grape'],
| weight: 340, gender: 'm'})
|
|
| {
|   acknowledged: true,
|   insertedIds: { '0': ObjectId('6a0ccb3ade48b0ea48abc124') }
| }
learn> db.unicorns.find({name: "Barney"})
[
| {
|   _id: ObjectId('6a0ccb3ade48b0ea48abc124'),
|   name: 'Barney',
|   loves: [ 'grape' ],
|   weight: 340,
|   gender: 'm'
| }
| ]
```

Рис. 36 - Вставка нового документа

Практическое задание 3.3.2.

- 1) Для самки единорога Ауна внести изменения в БД: теперь ее вес 800, она убила 51 вампира.

Метод update устарел, необходимо использовать updateOne или updateMany с обязательным использованием атомарных операторов.

```
db.unicorns.updateOne({name : "Ayna"}, {$set: {weight: 800, vampires:
51}})
```

Для обновления отдельных полей в этих функциях применяется оператор \$set. Если документ не содержит обновляемое поле, то оно создается.

```
learn> db.unicorns.updateOne({name : "Ayna"}, {$set: {weight: 800, vampires:
51}})
{
|   acknowledged: true,
|   insertedId: null,
|   matchedCount: 1,
|   modifiedCount: 1,
|   upsertedCount: 0
| }
```

Рис. 37 - Обновление документа

- 2) Проверим содержимое таблицы

```
learn> db.unicorns.find({name: 'Ayna'})
[
  {
    _id: ObjectId('6a0ca415de48b0ea48abc119'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 800,
    gender: 'f',
    vampires: 51
  }
]
```

Рис. 38 - Результат выполнения

Практическое задание 3.3.3.

- 1) Для самца единорога Raleigh внести изменения в БД: теперь он любит рэдбул.

```
db.unicorns.updateOne({name: "Raleigh"}, {$set:{loves: ['redbull']}})
```

```
learn> db.unicorns.updateOne({name: "Raleigh"}, {$set:{loves: ['redbull']}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Рис. 39 - Обновление документа

- 2) Проверить содержимое коллекции unicorns.

```
learn> db.unicorns.find({name: "Raleigh"})
[
  {
    _id: ObjectId('6a0ca415de48b0ea48abc11b'),
    name: 'Raleigh',
    loves: [ 'redbull' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  }
]
```

Рис. 40 - Результат выполнения

Практическое задание 3.3.4.

- 1) Всем самцам единорогов увеличить количество убитых вампиров на 5.

```
learn> db.unicorns.updateMany({gender: "m"}, {$inc:{vampires: 5}})
```

```
learn> db.unicorns.updateMany({gender: "m"}, {$inc:{vampires: 5}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 8,
  modifiedCount: 8,
  upsertedCount: 0
}
```

Рис. 41 - Результат выполнения

- 2) Проверить содержимое коллекции unicorns.

```
learn> db.unicorns.find()
[
  {
    _id: ObjectId('6a0ca415de48b0ea48abc114'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 68
  },
  {
    _id: ObjectId('6a0ca415de48b0ea48abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a0ca415de48b0ea48abc116'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 187
  },
  {
    _id: ObjectId('6a0ca415de48b0ea48abc117'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
  }
]
```

Рис. 42 - Результат выполнения

Практическое задание 3.3.5.

- 1) Изменить информацию о городе Портланд: мэр этого города теперь беспартийный.

```
db.towns.updateOne({name: "Portland"}, {$unset: {"mayor.party": 1}})
```

```
learn> db.towns.updateOne({name: "Portland"}, {$unset: {"mayor.party": 1}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Рис. 43 - Результат выполнения

2) Проверить содержимое коллекции towns.

```
learn> db.towns.find()
[
  {
    _id: ObjectId('6a0cbb24de48b0ea48abc121'),
    name: 'Punxsutawney ',
    populatiuon: 6200,
    last_sensus: ISODate('2008-01-31T00:00:00.000Z'),
    famous_for: [ ' ' ],
    mayor: { name: 'Jim Wehrle' }
  },
  {
    _id: ObjectId('6a0cbb53de48b0ea48abc122'),
    name: 'New York',
    populatiuon: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a0cbb69de48b0ea48abc123'),
    name: 'Portland',
    populatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams' }
  }
]
```

Рис. 44 - Результат выполнения

Практическое задание 3.3.6.

1) Изменить информацию о самце единорога Pilot: теперь он любит и шоколад.

```
db.unicorns.updateOne({name : "Pilot"}, {$push:
{loves: "chocolate"}}
```

```
learn> db.unicorns.updateOne({name : "Pilot"}, {$push: {loves: "chocolate"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Рис. 45 - Результат выполнения

2) Проверить содержимое коллекции unicorns.

```
learn> db.unicorns.find({name : "Pilot"})
[
  {
    _id: ObjectId('6a0ca415de48b0ea48abc11d'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon', 'chocolate' ],
    weight: 650,
    gender: 'm',
    vampires: 59
  }
]
```

Рис. 46 - Результат выполнения

Практическое задание 3.3.7.

1) Изменить информацию о самке единорога Aurora: теперь она любит еще и сахар, и лимоны.

```
db.unicorns.updateOne({name: "Aurora"}, {$addToSet : {loves: {$each : ['sugar', 'lemon']}}})
learn> db.unicorns.updateOne({name: "Aurora"}, {$addToSet : {loves: {$each : ['sugar', 'lemon']}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Рис. 47 - Результат выполнения

2) Проверить содержимое коллекции unicorns.

```
learn> db.unicorns.find({name : "Aurora"})
[
  {
    _id: ObjectId('6a0ca415de48b0ea48abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
```

Рис. 48 - Результат выполнения

3.4 УДАЛЕНИЕ ДАННЫХ ИЗ КОЛЛЕКЦИИ

Команда `remove` является устаревшей, вместо нее будут использоваться команды `deleteOne` или `deleteMany`.

Практическое задание 3.4.1.

- 1) Создайте коллекцию `towns`, включающую следующие документы:

```
{name: "Punxsutawney ",
popujatiuon: 6200,
last_sensus: ISODate("2008-01-31"),
famous_for: ["phil the groundhog"],
mayor: {
  name: "Jim Wehrle"
}}
{name: "New York",
popujatiuon: 22200000,
last_sensus: ISODate("2009-07-31"),
famous_for: ["status of liberty", "food"],
mayor: {
  name: "Michael Bloomberg",
party: "I"}}
{name: "Portland",
popujatiuon: 528000,
last_sensus: ISODate("2009-07-20"),
famous_for: ["beer", "food"],
mayor: {
  name: "Sam Adams",
party: "D"}}
```

Для начала я удалю имеющуюся коллекцию `towns`, сделать это можно с помощью команды `drop()`.

```
learn> db.towns.drop()
true
```

Рис. 49 - Удаление коллекции

```
learn> db.createCollection('towns')
{ ok: 1 }
```

Рис. 50 - Создание коллекции towns

```
learn> db.towns.find()
[
  {
    _id: ObjectId('6a0d74e09397112e1babc114'),
    name: 'Punxsutawney ',
    popujatiuon: 6200,
    last_sensus: ISODate('2008-01-31T00:00:00.000Z'),
    famous_for: [ 'phil the groundhog' ],
    mayor: { name: 'Jim Wehrle' }
  },
  {
    _id: ObjectId('6a0d74fc9397112e1babc115'),
    name: 'New York',
    popujatiuon: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a0d75169397112e1babc116'),
    name: 'Portland',
    popujatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
```

Рис. 51 - Вывод всех документов после добавления

2) Удалите документы с беспартийными мэрами.

```
db.towns.deleteMany({"mayor.party": {$exists:
false}})
```

```
learn> db.towns.deleteMany({"mayor.party": {$exists: false}})
{ acknowledged: true, _id: '6a0d74e09397112e1babc114', deletedCount: 1 }
```

Рис. 52 - Удаление по заданному условию

3) Проверьте содержание коллекции.


```
learn> db.towns.find()
[
  {
    _id: ObjectId('6a0d74fc9397112e1babc115'),
    name: 'New York',
    popujatiuon: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a0d75169397112e1babc116'),
    name: 'Portland',
    popujatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
```

Рис. 53 - Результат выполнения

4) Очистите коллекцию.

```
db.towns.deleteMany({})
learn> db.towns.deleteMany({})
{ acknowledged: true, deletedCount: 2 }
```

Рис. 54 - Результат выполнения

5) Просмотрите список доступных коллекций.

Коллекция towns доступна, но она пустая.

```
learn> show collections
towns
unicorns
learn> db.towns.find()
```

Рис. 55 - Результат выполнения

4 ССЫЛКИ И РАБОТА С ИНДЕКСАМИ В БАЗЕ ДАННЫХ MONGODB

4.1 ССЫЛКИ В БД

Практическое задание 4.1.1.

- 1) Создайте коллекцию зон обитания единорогов, указав в качестве идентификатора кратко название зоны, далее включив полное название и описание.

```
db.habitats.insertMany([
|   { _id: 'forest', name: 'Magical forest',
description: 'Magical forest with glowing plants'},
|   { _id: 'limbo', name: 'Limbo', description: 'A
space outside of time and a forbidden zone'},
|   { _id: 'lake', name: 'Crystal lake',
description: 'A lake where unicorns like to come for
summer holidays'}
| ])
{
```

```
learn> db.habitats.insertMany([
|   { _id: 'forest', name: 'Magical forest', description: 'Magical forest wi
th glowing plants'},
|   { _id: 'limbo', name: 'Limbo', description: 'A space outside of time and
a forbidden zone'},
|   { _id: 'lake', name: 'Crystal lake', description: 'A lake where unicorns
like to come for summer holidays'}
| ])
{
  acknowledged: true,
  insertedIds: { '0': 'forest', '1': 'limbo', '2': 'lake' }
}
```

Рис. 56 - Создание коллекции зон обитания

```

learn> show collections
habitats
towns
unicorns
learn> db.habitats.find()
[
  {
    _id: 'forest',
    name: 'Magical forest',
    description: 'Magical forest with glowing plants'
  },
  {
    _id: 'limbo',
    name: 'Limbo',
    description: 'A space outside of time and a forbidden zone'
  },
  {
    _id: 'lake',
    name: 'Crystal lake',
    description: 'A lake where unicorns like to come for summer holidays'
  }
]

```

Рис. 57 - Результат выполнения

- 2) Включите для нескольких единорогов в документы ссылку на зону обитания, используя второй способ автоматического связывания.

```

learn> db.unicorns.updateOne({name: 'Kenny'}, {$set : {habitat : {$ref: 'habitats', $id : 'forest'}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.updateOne({name: 'Unicrom'}, {$set : {habitat : {$ref: 'habitats', $id : 'limbo'}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}

```

Рис. 58 - Результат выполнения

- 3) Проверьте содержание коллекции единорогов.

```
learn> db.unicorns.find()
[
  {
    _id: ObjectId('6a0d7460bd07e0f3bfabc114'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a0d7460bd07e0f3bfabc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a0d7460bd07e0f3bfabc116'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
```

Рис. 59 - Просмотр содержимого коллекции

```

    _id: ObjectId('6a0d7460bd07e0f3bfabc11d'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54,
    habitat: DBRef('habitats', 'lake')
  },
  {
    _id: ObjectId('6a0d7460bd07e0f3bfabc11a'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39,
    habitat: DBRef('habitats', 'forest')
  },
]
```

Рис. 60 - Результат выполнения

Извлечем из коллекции данные о единорогах, которым я добавила место обитания. На примере единорога Pilot.

В моей версии mongosh свойство идентификатора доступно именно как `oid`, а не `id`.

```
learn> var pilot = db.unicorns.findOne({name: 'Pilot'})

learn> db[pilot.habitat.collection].findOne({_id: pilot.habitat.oid})
{
  _id: 'lake',
  name: 'Crystal lake',
  description: 'A lake where unicorns like to come for summer holidays'
}
{
  _id: ObjectId('6a0d7460bd07e0f3bfabc116'),
  name: 'Unicrom',
  loves: [ 'energon', 'redbull' ],
  weight: 984,
  gender: 'm',
  vampires: 182,
  habitat: DBRef('habitats', 'limbo')
},
```

Рис. 61 - Результат выполнения

4.2 НАСТРОЙКА ИНДЕКСОВ

Практическое задание 4.2.1.

- 1) Проверьте, можно ли задать для коллекции `unicorns` индекс для ключа `name` с флагом `unique`.

```
learn> db.unicorns.ensureIndex({"name": 1}, {"unique": true})
[ 'name_1' ]
```

Рис. 62 - Создание индекса

Индекс создан успешно. Можно было еще проверить посредством уникальности имен. Например,

```
learn> db.unicorns.aggregate({$group : {_id: "$name", count: {$sum: 1}}})
[
  { _id: 'Raleigh', count: 1 },
  { _id: 'Rooooooodles', count: 1 },
  { _id: 'Unicrom', count: 1 },
  { _id: 'Horny', count: 1 },
  { _id: 'Aurora', count: 1 },
  { _id: 'Solnara', count: 1 },
  { _id: 'Ayna', count: 1 },
  { _id: 'Kenny', count: 1 },
  { _id: 'Leia', count: 1 },
  { _id: 'Nimue', count: 1 },
  { _id: 'Pilot', count: 1 }
]
```

Рис. 63 - Просмотр уникальности имен

Как мы можем заметить - все имена уникальны.

4.3 УПРАВЛЕНИЕ ИНДЕКСАМИ

Практическое задание 4.3.1.

- 1) Получите информацию о всех индексах коллекции unicorns .

```
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
```

Рис. 64 - Получение индексов

- 2) Удалите все индексы, кроме индекса для идентификатора.

```
learn> db.unicorns.dropIndex("name_1")
{ nIndexesWas: 2, ok: 1 }
```

Рис. 65 - Удаление созданного индекса

Посмотрим информацию о всех индексах.

```
learn> db.unicorns.getIndexes()
[ { v: 2, key: { _id: 1 }, name: '_id_' } ]
```

Рис. 66 - Получение индексов

Остался только для идентификатора.

3) Попробуйте удалить индекс для идентификатора.

```
learn> db.unicorns.dropIndex("_id_")
MongoServerError[InvalidOptions]: cannot drop _id index
```

Рис. 67 - Результат выполнения

Удалить индекс идентификатора невозможно, потому что он является обязательным(выполняет роль первичного ключа). Чтобы избавиться от него, нужно удалить коллекцию или создать новую без системного индекса с помощью параметра `autoIndexId: false`.

4.4 ПЛАН ЗАПРОСА

1) Создайте объемную коллекцию `numbers`, задействовав курсор:

```
for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}
```

```
learn> var cursor = db.numbers.find()
```

```
learn> for(var i = 0; i < 100000; i++) {
|   db.numbers.insert({value: i})
| }
```

```
learn> for(var i = 0; i < 100000; i++) {
|   db.numbers.insert({value: i})
| }
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a0d910d9397112e1baef3d7') }
}
```

Рис. 68 - Создание коллекции `numbers`

```
learn> db.numbers.find()
[
  { _id: ObjectId('6a0d8dfd9397112e1bac2217'), value: 0 },
  { _id: ObjectId('6a0d8dfd9397112e1bac2218'), value: 1 },
  { _id: ObjectId('6a0d8dfd9397112e1bac2219'), value: 2 },
  { _id: ObjectId('6a0d8dfd9397112e1bac221a'), value: 3 },
  { _id: ObjectId('6a0d8dfd9397112e1bac221b'), value: 4 },
  { _id: ObjectId('6a0d8dfd9397112e1bac221c'), value: 5 },
  { _id: ObjectId('6a0d8dfd9397112e1bac221d'), value: 6 },
  { _id: ObjectId('6a0d8dfd9397112e1bac221e'), value: 7 },
  { _id: ObjectId('6a0d8dfd9397112e1bac221f'), value: 8 },
  { _id: ObjectId('6a0d8dfd9397112e1bac2220'), value: 9 },
  { _id: ObjectId('6a0d8dfd9397112e1bac2221'), value: 10 },
  { _id: ObjectId('6a0d8dfd9397112e1bac2222'), value: 11 },
  { _id: ObjectId('6a0d8dfd9397112e1bac2223'), value: 12 },
  { _id: ObjectId('6a0d8dfd9397112e1bac2224'), value: 13 },
  { _id: ObjectId('6a0d8dfd9397112e1bac2225'), value: 14 },
  { _id: ObjectId('6a0d8dfd9397112e1bac2226'), value: 15 },
  { _id: ObjectId('6a0d8dfd9397112e1bac2227'), value: 16 },
  { _id: ObjectId('6a0d8dfd9397112e1bac2228'), value: 17 },
  { _id: ObjectId('6a0d8dfd9397112e1bac2229'), value: 18 },
  { _id: ObjectId('6a0d8dfd9397112e1bac222a'), value: 19 }
]
Type "it" for more
```

Рис. 69 - Вывод содержимого коллекции

Эта вставка очень долго выполнялась.

2) Выберите последних четыре документа.

```
learn> db.numbers.find().sort({value: -1}).limit(4)
[
  { _id: ObjectId('6a0d910d9397112e1baef3d7'), value: 99999 },
  { _id: ObjectId('6a0d910d9397112e1baef3d6'), value: 99998 },
  { _id: ObjectId('6a0d910d9397112e1baef3d5'), value: 99997 },
  { _id: ObjectId('6a0d910d9397112e1baef3d4'), value: 99996 }
]
```

Рис. 70 - Результат выполнения

3) Проанализируйте план выполнения запроса 2. Сколько потребовалось времени на выполнение запроса? (по значению параметра `executionTimeMillis`)


```

learn> db.numbers.find().sort({value: -1}).limit(4).explain("executionStats"
)
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: 'AE5753F2',
    planCacheShapeHash: 'AE5753F2',
    planCacheKey: 'AE69D0D4',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'SORT',
      sortPattern: { value: -1 },
      memLimit: 104857600,
      limitAmount: 4,
      type: 'simple',
      inputStage: { stage: 'COLLSCAN', nss: 'learn.numbers', direction: 'for
ward' }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 145,
    totalKeysExamined: 0,
    totalDocsExamined: 100000,
    executionStages: {
      isCached: false,
      stage: 'SORT',
      nReturned: 4,

```

Рис. 71 - Результат выполнения

Потребовалось 145 миллисекунд.

- 4) Создайте индекс для ключа value и получите информацию о всех индексах коллекции numbers.

```

learn> db.numbers.createIndex({value: 1})
value_1
learn> db.numbers.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]

```

Рис. 72 - Результат выполнения

5) Выполните запрос 2.

```
learn> db.numbers.find().sort({value: -1}).limit(4)
[
  { _id: ObjectId('6a0d910d9397112e1baef3d7'), value: 99999 },
  { _id: ObjectId('6a0d910d9397112e1baef3d6'), value: 99998 },
  { _id: ObjectId('6a0d910d9397112e1baef3d5'), value: 99997 },
  { _id: ObjectId('6a0d910d9397112e1baef3d4'), value: 99996 }
]
```

Рис. 73 - Результат выполнения

Вывод не изменился.

6) Проанализируйте план выполнения запроса с установленным индексом. Сколько потребовалось времени на выполнение запроса?

```
learn> db.numbers.find().sort({value: -1}).limit(4).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: 'AE5753F2',
    planCacheShapeHash: 'AE5753F2',
    planCacheKey: 'AE69D0D4',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'LIMIT',
      limitAmount: 4,
      inputStage: {
        stage: 'FETCH',
        nss: 'learn.numbers',
        inputStage: {
          stage: 'IXSCAN',
          nss: 'learn.numbers',
          keyPattern: { value: 1 },
          indexName: 'value_1',
          isMultiKey: false,
          multiKeyPaths: { value: [] },
          isUnique: false,
          isSparse: false,
          isPartial: false,
          indexVersion: 2,
          direction: 'backward',
          indexBounds: { value: [ '[MaxKey, MinKey]' ] }
        }
      }
    }
  },
}
```

Рис. 74 - Результат выполнения

Из вывода - executionTimeMillis: 2, т.е. при создании индексов время выполнения запроса у нас уменьшилось(стало 2 миллисекунды).

- 7) Сравните время выполнения запросов с индексом и без. Дайте ответ на вопрос: какой запрос более эффективен?

Время выполнения при 1 запросе : 145 мс

Время выполнения при 2 запросе : 2 мс

Значительно эффективнее запрос с индексами, так как быстрее в 72.5 раза.

ВЫВОД

В данной лабораторной работе я овладела практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками в базе данных MongoDB.